Fault-Tolerance Scaffold Simulator

Hardware Errors, Measurement Errors, and Software Decoding

Anchor: Classical repetition-code scaffold for introducing the systems logic of quantum error correction.

Purpose: Demystify fault tolerance as a hardware–measurement–software co-design problem before introducing full quantum error-correction machinery.

Boris Kiefer
New Mexico State University
bkiefer@nmsu.edu
GitHub
LinkedIn

Abstract

Fault-tolerant quantum computing is often introduced through specialized language: logical qubits, syndrome measurements, stabilizer codes, decoders, thresholds, and physical error rates. This brief presents a deliberately classical simulator that exposes the core systems idea before introducing quantum mechanics. A logical bit is encoded into an odd-length string of physical bits. Hardware errors flip physical bits with probability p, measurement errors corrupt the software-visible readout with probability q, and a majority-vote decoder attempts to recover the logical value. The simulator illustrates three central lessons: redundancy plus software decoding can suppress logical failure; $p = 0.5$ marks a total-randomness boundary for majority voting; and practical operation is governed not by that generous information limit, but by stricter logical-error targets such as 10^{-2}, 10^{-3}, or lower.

> **Core message.** Fault tolerance is not hardware alone and not software alone. It is the co-design of hardware error rates, measurement fidelity, decoder logic, and corrective action.

Introduction

Quantum computing is moving toward a fault-tolerance-centered narrative. A useful quantum computer is not built from perfect physical qubits. Instead, reliable logical behavior must emerge from imperfect hardware, repeated measurements, classical decoding, and corrective control. The threshold theorem formalizes this idea: under suitable assumptions, error correction can suppress logical errors when physical error rates are below an appropriate threshold [1]. Stabilizer codes and surface-code architectures provide practical frameworks for this goal [2, 3].

The current state of the field reinforces this systems view. Recent surface-code experiments have shown logical-error suppression through code scaling and below-threshold operation with real-time decoding [7, 8]. Industry roadmaps and architecture papers increasingly frame progress around logical qubits, real-time decoding, compiler/runtime integration, micro-architecture, and full-stack fault tolerance rather than only physical-qubit counts [10, 11, 9].

However, fault tolerance can appear opaque to students, researchers in adjacent fields, and technical stakeholders. The present simulator therefore starts from a simpler classical analogue: repeated bits and majority voting. This demystifies the systems logic behind fault tolerance while avoiding the complications of quantum measurement, no-cloning, phase errors, leakage, and syndrome extraction.

Theory

Classical repetition scaffold

A logical bit $b \in \{0, 1\}$ is encoded into an odd-length string of N physical bits,

$$b \longrightarrow \underbrace{bb\cdots b}_{N \text{ bits}}, \qquad N = 1, 3, 5, \ldots \tag{1}$$

Each physical bit is independently flipped by the hardware with probability p. Each measured bit is then independently corrupted by readout/measurement error with probability q:

$$\text{logical bit} \to \text{hardware error} \to \text{measurement error} \to \text{software decoder}.$$

Under independent symmetric errors, the probability that the software observes a wrong bit is

$$r = p(1-q) + (1-p)q = p + q - 2pq. \tag{2}$$

The majority-vote decoder fails when more than half of the observed bits are wrong. Therefore, for odd N,

$$p_L(N, p, q) = \sum_{k=(N+1)/2}^{N} \binom{N}{k} r^k (1-r)^{N-k}, \tag{3}$$

where p_L is the logical failure probability.

For $N = 1$, there is no redundancy and no meaningful majority protection. With $q = 0$, Eq. (3) gives $p_L = p$ exactly. Thus, the $N = 1$ curve is the unencoded baseline: it lies on the $p_L = p$ reference line when measurement is ideal. When $q > 0$, the decoder sees the effective observed error $r = p + q - 2pq$, so the single-bit logical failure becomes $p_L = r$ rather than p.

Information limit versus useful operating regime

For the idealized independent-error model with $q < 0.5$, the majority-vote information boundary occurs at

$$p = 0.5. \tag{4}$$

At this point each physical bit is random relative to the original encoded bit. Majority voting has no usable information and is equivalent to guessing.

This point is important, but it is not the practical engineering target. Useful operation requires logical failure rates far below the information limit. The simulator therefore distinguishes three conditions:

- **Information boundary:** $p = 0.5$, where majority voting loses information.
- **Improvement boundary:** $p_L = p$, where encoding/decoding is no better than using one physical bit.
- **Application target:** $p_L < p_{\text{target}}$, such as 10^{-2}, 10^{-3}, or lower depending on workload.

Simulator Knobs and Dials

The simulator is intentionally small. Its controls expose the minimal hardware–software trade space:

Control	Meaning	Role in the simulator
N	String length	Redundancy level; restricted to odd values.
p	Hardware bit-flip probability	Physical error channel before measurement.
q	Measurement/readout error probability	Corrupts the evidence seen by the decoder.
trials	Monte Carlo samples	Compares sampling to the analytical binomial result.
$p_{\max}, q_{\max}$	Heatmap axes	Zoom into the useful lower-error operating region.
p_{target}	Logical-error contours	User-selected performance goals.

> The line plot shows the full conceptual threshold structure. The heatmap is a zoomable engineering view. This prevents the generous $p = 0.5$ information limit from obscuring the much smaller region where useful logical-error targets are achieved.

Simulator Interpretation

Figure 1 summarizes the two main simulator views. Panel (a) shows the broad majority-vote behavior. For $p < 0.5$, each bit is more likely correct than wrong, so increasing N can suppress logical failure. At $p = 0.5$, each bit is random and the decoder has no information. For $p > 0.5$, majority voting becomes systematically misleading.

Panel (b) is the useful operating-region view. For fixed N, the heatmap uses

$$x = p, \qquad y = q, \qquad \text{color} = p_L(N, p, q).$$

The red contours mark user-selected logical-error targets. The black dashed contour marks $p_L = p$, the boundary where encoding stops improving over one physical bit. The axes are adjustable because small logical-error targets are concentrated in the lower-left region of the (p, q) plane.

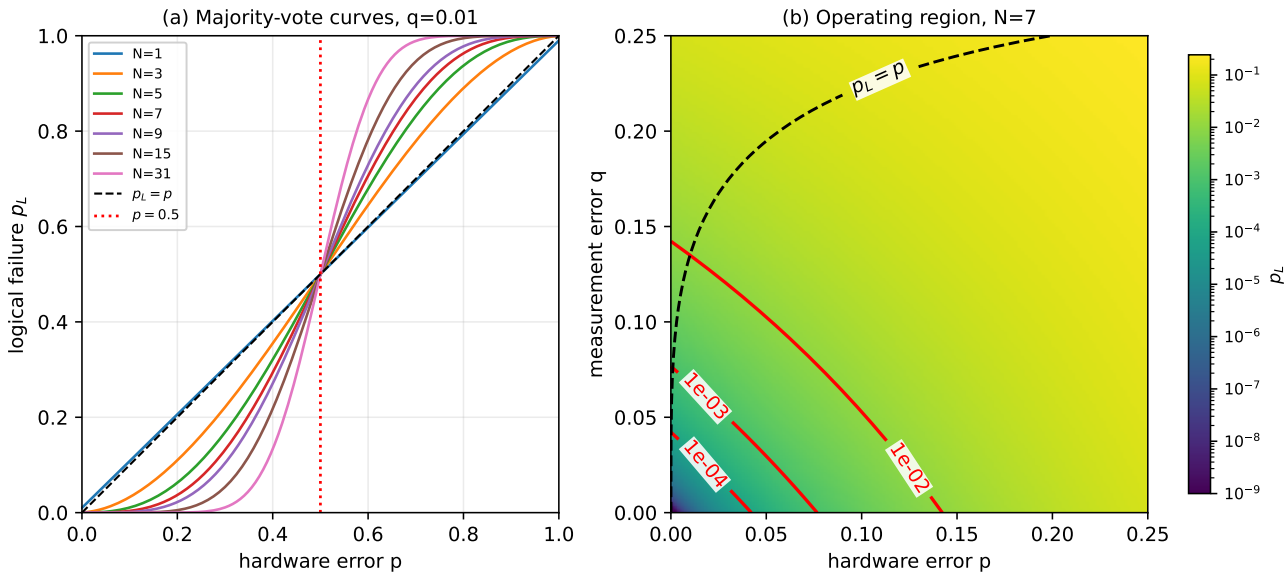

Figure 1: Fault-tolerance scaffold simulator. (a) Majority-vote logical failure curves for several repetition lengths N at fixed measurement error q. The dashed diagonal $p_L = p$ is the unencoded reference, and the vertical line at $p = 0.5$ marks the total-randomness/information-loss boundary of the majority-vote model. For $N = 1$ and $q = 0$, the curve lies on $p_L = p$ because no redundancy is available. (b) Zoomable hardware–measurement operating region for fixed N. The horizontal axis is hardware bit-flip probability p, the vertical axis is measurement/readout error probability q, and color indicates logical failure probability p_L. Red contours mark user-selected logical-error targets; the black dashed contour marks $p_L = p$, where encoding stops improving over a single physical bit.

Connection to Quantum Error Correction

The simulator is intentionally classical, but the analogy to quantum error correction is direct:

Classical scaffold	Quantum error-correction analogue
Physical bit flip	Physical qubit error
Readout error q	Noisy syndrome measurement
Majority vote	Decoder / inference algorithm
Corrected logical bit	Logical qubit / Pauli-frame update
Logical failure p_L	Logical error per round or operation

The important difference is that quantum information cannot simply be copied and directly measured. Quantum codes use syndrome measurements to infer error information without directly measuring and destroying the protected logical state. Stabilizer-code theory provides the formal framework for this process [2]. Surface-code architectures extend this idea into a hardware-aware layout where local measurements, repeated syndrome extraction, and classical decoding jointly determine logical reliability [3].

Modern demonstrations and architecture proposals now emphasize not only code distance, but also real-time decoding, correlated error events, compiler/runtime integration, micro-architecture, and operation beyond break-even [4, 5, 8, 9]. Thus, the simulator should be read as a scaffold, not a replacement for a quantum code. It teaches the first systems principle: reliable logical behavior emerges from physical redundancy, diagnostic information, and software decoding.

Takeaway

The simulator makes fault tolerance accessible by separating the architecture into three layers:

- **Hardware layer:** physical bits fail with probability p.
- **Measurement layer:** the software observes imperfect evidence with error probability q.

- **Software layer:** a decoder converts noisy evidence into a logical decision.

The point $p = 0.5$ is not a practical performance goal; it is the total-randomness boundary where majority vote loses information. Practical fault tolerance is governed by much stricter logical-error targets. This distinction is the key lesson: fault tolerance is a hardware–software co-design problem, and useful operation depends on the combined performance of hardware errors, measurement fidelity, decoder logic, and application-level error requirements.

Acknowledgment This material was developed and/or adapted with support from the National Science Foundation through the QCAP-Pilot and QCAP-Design efforts under NSF Award Nos. OSI-2410813 and OSI-2531569. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

References

[1] D. Aharonov and M. Ben-Or, *Fault-Tolerant Quantum Computation With Constant Error Rate*, arXiv:quant-ph/9906129, 1999.

[2] D. Gottesman, *Stabilizer Codes and Quantum Error Correction*, Ph.D. thesis, California Institute of Technology, arXiv:quant-ph/9705052, 1997.

[3] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, *Surface codes: Towards practical large-scale quantum computation*, Physical Review A **86**, 032324, 2012.

[4] N. Ofek, A. Petrenko, R. Heeres, P. Reinhold, Z. Leghtas, B. Vlastakis, Y. Liu, L. Frunzio, S. M. Girvin, L. Jiang, M. H. Devoret, and R. J. Schoelkopf, *Extending the lifetime of a quantum bit with error correction in superconducting circuits*, Nature **536**, 441–445, 2016.

[5] V. V. Sivak et al., *Real-time quantum error correction beyond break-even*, Nature **616**, 50–55, 2023.

[6] Z. Chen et al., *Exponential suppression of bit or phase errors with cyclic error correction*, Nature **595**, 383–387, 2021.

[7] R. Acharya et al., *Suppressing quantum errors by scaling a surface code logical qubit*, Nature **614**, 676–681, 2023.

[8] R. Acharya et al., *Quantum error correction below the surface code threshold*, Nature **638**, 920–926, 2025.

[9] F. Tripier, W. C. Chung, J. Young, S. Alam, B. Bjork, A. Brodutch, F. L. Buessen, N. J. Coble, T. Dellaert, D. Maslov, M. Roetteler, E. Tham, M. Webster, M. Ye, J. Gamble, A. Maksymov, J. P. Marceaux, and N. Delfosse, *Fault-Tolerant Quantum Computing with Trapped Ions: The Walking Cat Architecture*, arXiv:2604.19481, 2026.

[10] IBM Quantum, *How IBM will build the world's first large-scale, fault-tolerant quantum computer*, IBM Quantum Blog, 2025.

[11] Riverlane, *Riverlane publishes QEC Technology Roadmap*, Riverlane Press Release, 2024.

[12] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press, 10th Anniversary Edition, 2010.